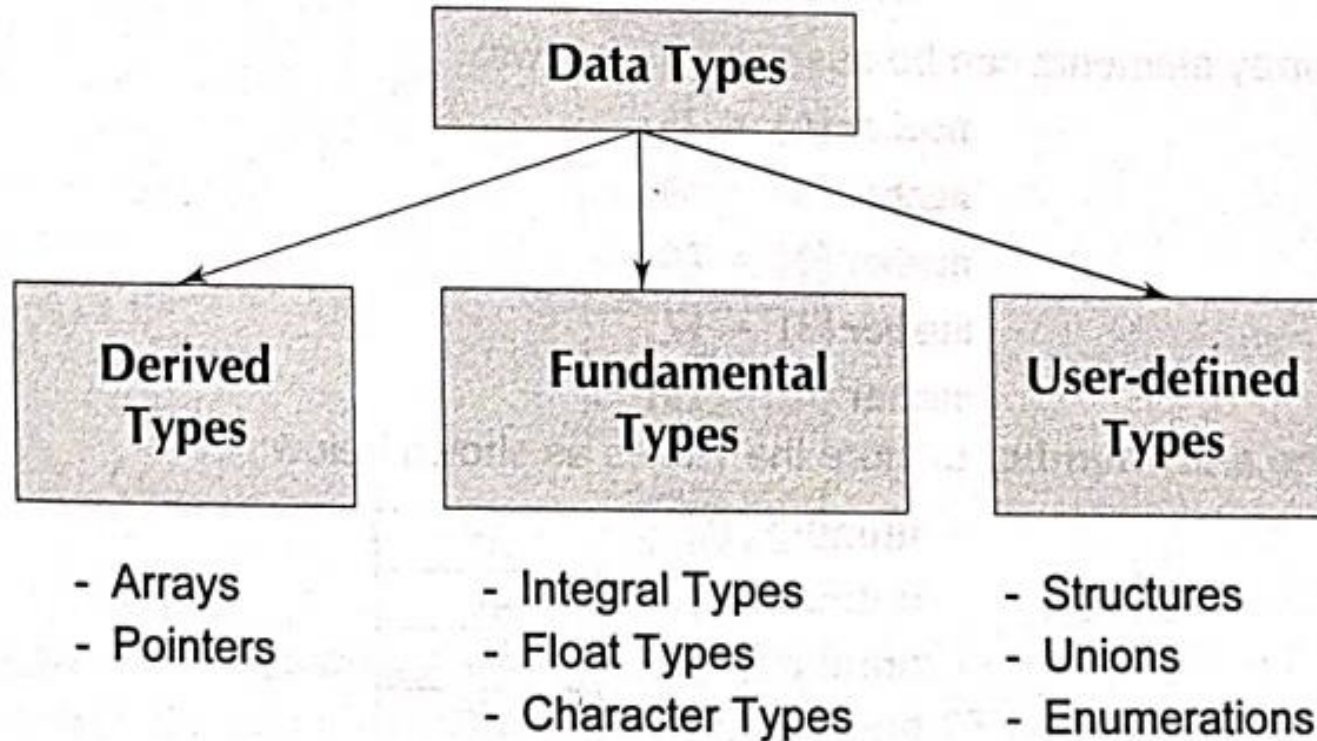


Chapter-7

Arrays

Data Types In C



Why?

➤ Consider the following Program,

```
#include <stdio.h>
void main ()
{
    int x;
    x=5;
    x=10;
    printf("x=%d", x);
}
```

What will be the value of x?

Cont...

- Variables are capable of holding only one value at a time.
- So far we have used only the fundamental data types, namely char, int, float, double and variations of int and double.
- Although these types are very useful, but a variable of these types can store only one value at any given time.
- So, they can be used only to handle limited amounts of data.
- In many applications, however, we need to handle a large volume of data in terms of reading, processing and printing.

Cont...

- However, there are a situation in which we want to store more than one value at a time in a single variable.
 - Suppose we wish to arrange the percentage marks obtained by 100 students in ascending order.
- We have two options to store these marks in memory.
 - a) Declare 100 variables to store percentage marks obtained by 100 different students, i.e. each variable containing one student's marks.
 - b) Construct array(subscripted variable) capable of storing all hundred values.

Examples where the concept of an array can be used.

- List of temperatures recorded every hour in a day, or a month, or a year.
- List of employees in an organization.
- List of products and their cost sold by a store.
- Test scores of a class of students.
- List of customers and their telephone numbers.
- Table of daily rainfall data.

Introduction to arrays

➤ An array is a fixed size sequenced collection of elements of the same data type and share a common name.

- **Types of Arrays**

- One Dimensional Arrays

- Two Dimensional Arrays

- Multidimensional Arrays

One-dimensional array

- A list of items can be given one variable name using only one subscript and such a variable is called a one-dimensional array.

Declaration of One-dimensional array,

General form

type variable-name [size];

- Type specifies the type of element that will be contained in the array.

ex. int, float, char etc.

Cont...

- size – is the maximum number of elements that can be stored inside the array.
- The subscript or index of an array can begin with number 0.
- That is `x[0]` is allowed. And refers to the 1st element of the array `x[10]`.
- We can define an array name **per** to represent a set of percentage marks of a group of 100 students.

i.e. **`float per[100];`**

Cont...

- Particular value is indicated by writing a number called index or subscript in brackets after the array name.

For example,

per[9]

Represents the percentage marks of 10th student.

Example

- If we want to represent a set of five numbers with an array variable `number` then first we have to declare it as:

```
int number[5];
```

- The computer reserves five storage locations as,

2000		number[0]
2002		number[1]
2004		number[2]
2006		number[3]
2008		number[4]

Cont...

➤ Value to the array elements can be assigned as:

number[0] = 19;

number[1] = 40;

number[2] = 20;

number[3] = 57;

number[4] = 19;

Cont...

- This would cause the array `number` to store the values as:

2000	19	<code>number[0]</code>
2002	40	<code>number[1]</code>
2004	20	<code>number[2]</code>
2006	57	<code>number[3]</code>
2008	19	<code>number[4]</code>

Cont...

- These elements may be used in programs just like any other C variable.

```
a = number[0] + 10;  
number[4] = number[1] + number[2];
```

- The index of an array can be integer constants or any expressions that results in integer.

Advantage of arrays

- The ability to use a single name to represent a collection of items.
- So items can be referred by specifying the item number.
- For example, a loop with the index of array as control variable can be used to
 - read the entire array.
 - perform calculation and print out the results.

Bound checking

- C performs no bound checking for array and therefore, care should be taken to ensure that the array index is within the declared limits.
- Because any reference to the arrays outside the declared limits would not necessarily cause an error. Rather, it might result in unpredictable program results.

Initialization of array

- **Compile time initialization**

➤ Initialize the elements of array when they are declared,

type arrayname[size]={list of values};

ex. `int arr[5]={ 10,20,25,30,40};`

`int arr[5]={ 1,2,3};`

then the remaining two elements will initialize to zero.

Cont...

- The size may be omitted. In such cases, compiler allocates enough space for all initialized elements

ex. **int counter[]={1,1,1,1};**

will declare the counter array to contain four elements with initial values 1.

- **int number [3] = { 10,20,30,40};**
Compiler will produce an error.

Run Time Initialization

```
int arr[100];
```

➤ First 50 to zero and last 50 to one.

```
for (i=0;i<50;i++)  
{  
    arr[i] = 0;  
}  
for (i=50;i<100;i++)  
{  
    arr[i] = 1;  
}
```

Cont...

- We can also use a scanf to initialize an array,

```
int x[2];  
scanf( "%d %d",&x[0] ,&x[1] );
```

Program: Print array elements

```
void main()
{
    int a[5]={11,12,13,14,15}; // {11,12,13};
    int i;
    /*printf("%d\n",a[0]);
    printf("%d\n",a[1]);
    printf("%d\n",a[2]);
    printf("%d\n",a[3]); //0
    printf("%d",a[4]); */ //0

    for(i=0;i<5;i++)
    {
        printf("%d\n",a[i]);
    }
}
```

Program : Read array elements

```
void main()
{
    int a[5];
    int i;
    /* scanf("%d",&a[0]);
    scanf("%d",&a[1]);
    scanf("%d",&a[2]);
    scanf("%d",&a[3]);
    scanf("%d",&a[4]); */

    for(i=0;i<5;i++)    // for(i=0;i<4;i++)
    {
        scanf("%d",&a[i]);
    }

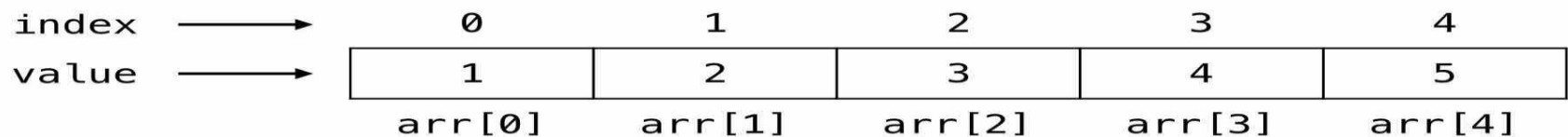
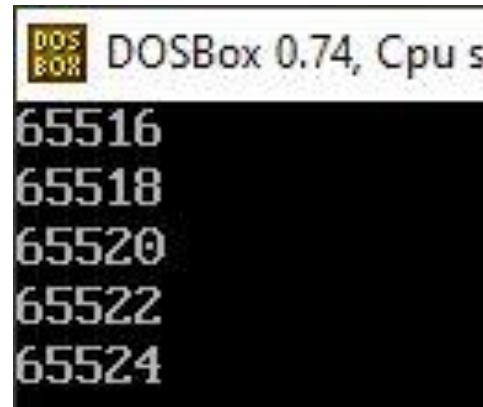
    //printf("%d",a[4]); //garbage value
}
```

Program: Size of array & array element

```
void main()
{
    int a[5];
    printf("%d",sizeof(a));    //10 bytes
    printf("%d",sizeof(a[2])); //2 bytes
}
```

Program: Address of Array

```
void main()
{
    int arr[5]={1,2,3,4,5};
    int i;
    for(i=0;i<5;i++)
    {
        printf("%u\n",&arr[i]);
    }
}
```



Size of array

```
void main()
{
    int a[10];
}
```

```
#define M 20
void main()
{
    int a[M];
}
```

```
#define M 20
#define N 10
void main()
{
    int a[M+N*2];
}
```

```
void main()
{
    int n;
    int a[n];
    scanf("%d",&n);
}
//Not valid.
//Supported by gcc compiler
```

Program: Sum and Average

```
#include<stdio.h>
void main()
{
    int a[5],i;
    float avg;
    int sum=0;
    for(i=0;i<5;i++)
    {
        scanf("%d",&a[i]);
    }
    for(i=0;i<5;i++)
    {
        printf("%d\n",a[i]);
    }
```

```
for(i=0;i<5;i++)  
{  
    sum=sum+a[i];  
}  
avg=(float)sum/5;  
printf("sum=%d",sum);  
printf("avg=%f",avg);  
}
```

Search Array Element.

```
#include<stdio.h>
void main()
{
    int a[10];
    int i,n,b;
    int pos,match=0;
    printf("enter number of elements:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
```

```
printf("enter element to be search:");
scanf("%d",&b);
for(i=0;i<n;i++)
{
    if(a[i]==b)
    {
        match=1;
        pos=i;
        printf("position=%d\n",i);
        // break;
    }
}
if(match==0)
{
    printf("no match found");
}
}
```

swap max and min number

```
#include<stdio.h>
void main()
{
    int a[10],i,max,min;
    int pos1=0,pos2=0;
    int temp,n;

    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
```

```
max=a[0];
```

```
min=a[0];
```

```
for(i=1;i<n;i++)
```

```
{
```

```
    if(a[i]>max)
```

```
    {
```

```
        max=a[i];
```

```
        pos1=i;
```

```
    }
```

```
    if(a[i]<min)
```

```
    {
```

```
        min=a[i];
```

```
        pos2=i;
```

```
    }
```

```
}
```

```
printf("min=%d",min);  
printf("max=%d\n",max);
```

```
temp=a[pos1];  
a[pos1]=a[pos2];  
a[pos2]=temp;
```

```
for(i=0;i<n;i++)  
{  
    printf("%d\n",a[i]);  
}  
}
```


Insert Array Element.

```
#include<stdio.h>

void main()
{
    int a[10],i,n,b,pos;

    printf("enter number of elements:");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }

    printf("enter element to be insert:");
    scanf("%d",&b);
```

```
printf("position of element:");  
scanf("%d",&pos);
```

```
for(i=n;i>pos;i--)  
{  
    a[i]=a[i-1];  
}
```

```
a[pos]=b;
```

```
printf("new array after insertion is:\n");  
for(i=0;i<=n;i++)  
{  
    printf("%d\n",a[i]);  
}  
}
```

More Programs.....(In Lab Session)

- Delete Array Element.
- Sort Elements of Array.

Two-dimensional array

➤ Situations where a table of values will have to be stored.

Student	Mat	Phy	Chem.
Student # 1	89	77	84
Student # 2	98	89	80
Student # 3	75	70	82
Student # 4	60	75	80
Student # 5	84	80	75

Declaration of 2-D Array

➤ General form

type arrayname[row_size][column_size];

`int stud[4][2];`

- First subscript of the variable stu is row number which changes for every student.
- Second subscript contains two columns,
 - zeroth column which contains rollno
 - first column which contains marks

Cont...

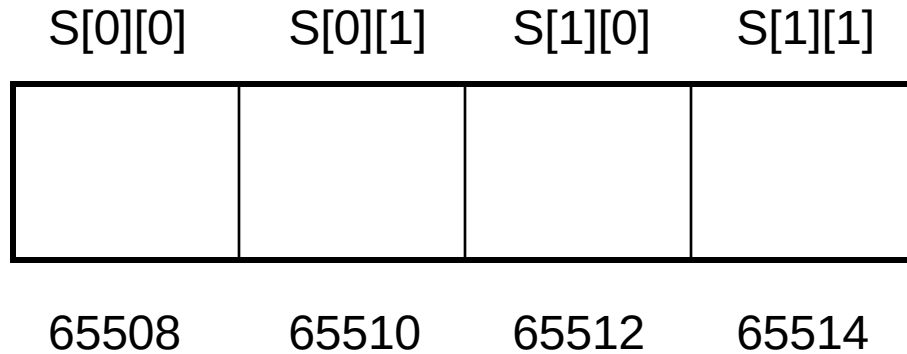
- Complete array arrangement is shown below.

	0	1
0	Stud[0][0]	Stud[0][1]
1	Stud[1][0]	Stud[1][1]
2	Stud[2][0]	Stud[2][1]
3	Stud[3][0]	Stud[3][1]

12	56
13	33
14	80
15	78

Memory map of 2-dimensional array

```
int s[2][2];
```



Initializing 2-dimensional array

➤ General format

type arrayname[row-size][col-size] = {list of values};

➤ `int a[2][3] = { 0,0,0,1,1,1 };`

initialize the elements of first row to 0 and second row to 1.

- The above statement can be equivalently written as

```
int a[2][3]= { { 0,0,0} , { 1,1,1} };
```

- We can also initialize a two dimensional array in the form of a matrix as,

```
int a[2][3]= {  
                { 0,0,0} ,  
                { 1,1,1}  
            };
```

- When the array is **completely initialized with all values**, explicitly we need not specify the size of first dimension.

```
int a [ ] [3] = { { 0,0,0} , { 1,1,1} };
```

- When all elements are to be initialized to zero, shortcut method may be used,

```
int m [3] [5] = { { 0 } , { 0 } , { 0 } };    OR    = {0};
```

Run time:-

```
void main()
{
    int a[2][2];
    int i,j;
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
}
```

	0	1
0	a[0][0]	a[0][1]
1	a[1][0]	a[1][1]

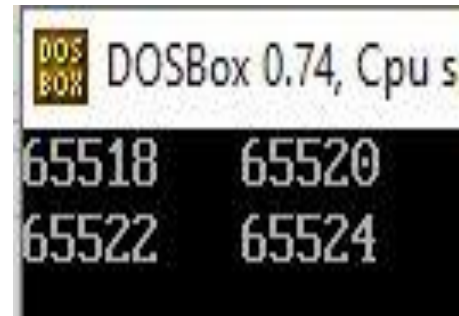
1	2
3	4

Print Elements of 2D array

```
for(i=0;i<2;i++)  
{  
    for(j=0;j<2;j++)  
    {  
        printf("%d\t",a[i][j]);  
    }  
    printf("\n");  
}
```

Program: Address of 2D Array

```
void main()
{
    int a[2][2];
    int i,j;
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            printf("%u\t",&a[i][j]);
        }
        printf("\n");
    }
}
```



Program: Addition of two 3 X 3 matrix.

	0	1	2
0	a[0][0]	a[0][1]	a[0][2]
1	a[1][0]	a[1][1]	a[1][2]
2	a[2][0]	a[2][1]	a[2][2]

	0	1	2
0	b[0][0]	b[0][1]	b[0][2]
1	b[1][0]	b[1][1]	b[1][2]
2	b[2][0]	b[2][1]	b[2][2]

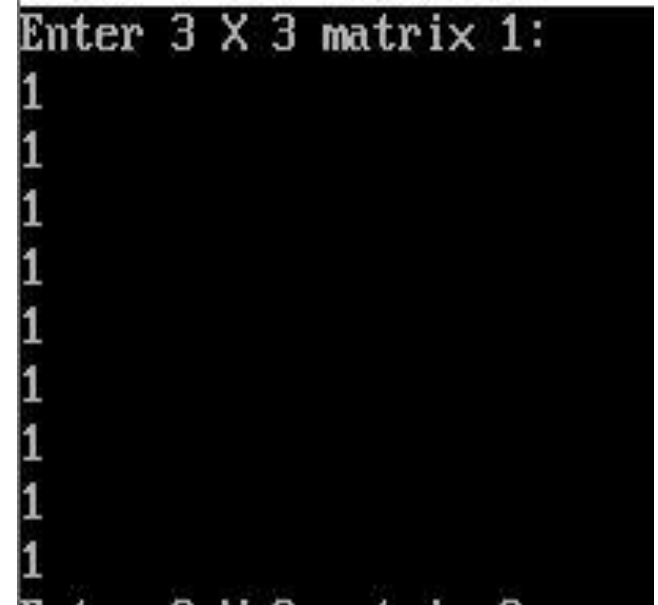
	0	1	2
0	c[0][0]	c[0][1]	c[0][2]
1	c[1][0]	c[1][1]	c[1][2]
2	c[2][0]	c[2][1]	c[2][2]

Program: Addition of two 3 X 3 matrix.

```
void main()
{
    int a[3][3],b[3][3],c[3][3],i,j;

    printf("Enter 3 X 3 matrix 1:\n");

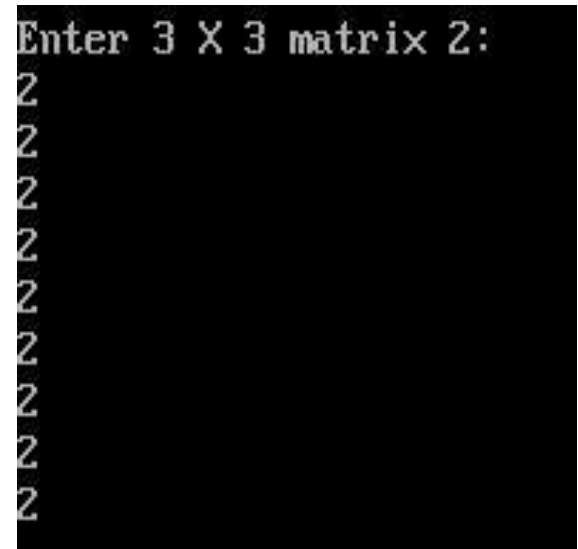
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
}
```



A screenshot of a terminal window with a black background and white text. The prompt "Enter 3 X 3 matrix 1:" is displayed at the top. Below the prompt, there is a vertical column of nine '1' characters, representing the input for the first matrix. The input is aligned to the left of the terminal window.

```
printf("Enter 3 X 3 matrix 2:\n");
```

```
for(i=0;i<3;i++)  
{  
    for(j=0;j<3;j++)  
    {  
        scanf("%d",&b[i][j]);  
    }  
}
```



```
Enter 3 X 3 matrix 2:  
2  
2  
2  
2  
2  
2  
2  
2  
2
```

```
for(i=0;i<3;i++)  
{  
    for(j=0;j<3;j++)  
    {  
        c[i][j]=a[i][j]+b[i][j];  
  
        //    printf("%d\t",c[i][j]);  
    }  
  
    //    printf("\n");  
}
```



```
printf("\n\nSum of Matrix\n\n");
```

```
for(i=0;i<3;i++)
```

```
{
```

```
    for(j=0;j<3;j++)
```

```
    {
```

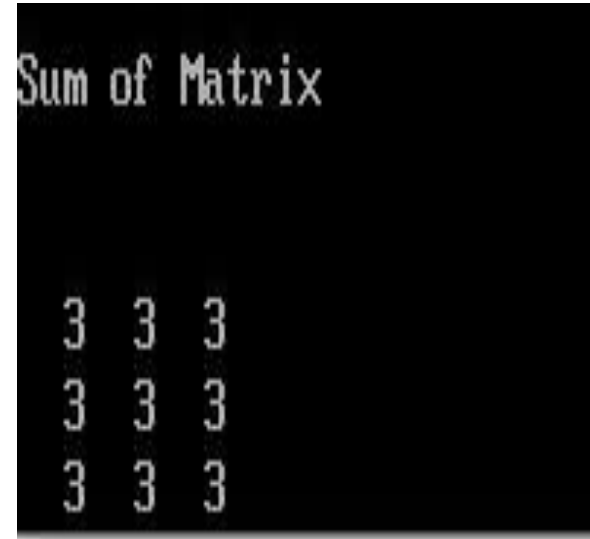
```
        printf("%d",c[i][j]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
}
```



```
Sum of Matrix

3 3 3
3 3 3
3 3 3
```

Program: Multiplication of two 3 X 3 matrix.

	0	1	2
0	a[0][0]	a[0][1]	a[0][2]
1	a[1][0]	a[1][1]	a[1][2]
2	a[2][0]	a[2][1]	a[2][2]

	0	1	2
0	b[0][0]	b[0][1]	b[0][2]
1	b[1][0]	b[1][1]	b[1][2]
2	b[2][0]	b[2][1]	b[2][2]

	0	1	2
0	c[0][0]	c[0][1]	c[0][2]
1	c[1][0]	c[1][1]	c[1][2]
2	c[2][0]	c[2][1]	c[2][2]

Multiplication of two 3 X 3 Matrix

```
for(i=0;i<3;i++)  
{  
    for(j=0;j<3;j++)  
    {  
        c[i][j]=0;  
        for(k=0;k<3;k++)  
            c[i][j]=c[i][j]+a[i][k]*b[k][j];  
    }  
}
```

Multiplication Formula

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

$$= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} & a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} & a_{11}b_{13} + a_{12}b_{23} + a_{13}b_{33} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} & a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32} & a_{21}b_{13} + a_{22}b_{23} + a_{23}b_{33} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} & a_{31}b_{12} + a_{32}b_{22} + a_{33}b_{32} & a_{31}b_{13} + a_{32}b_{23} + a_{33}b_{33} \end{bmatrix}$$

Multi-Dimensional array

➤ General form

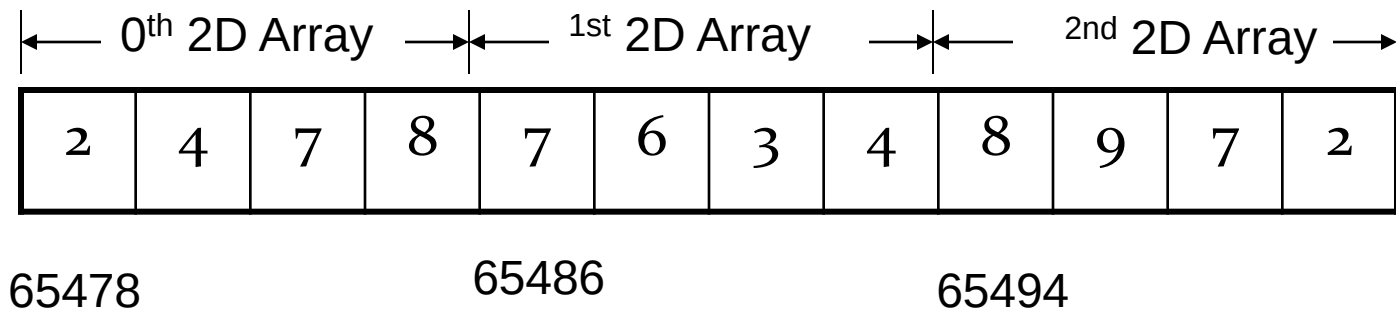
type array_name [s1] [s2] [s3][sm];

int arr [3] [2] [2];

➤ arr is a three-dimensional array contain 12 integer type elements.

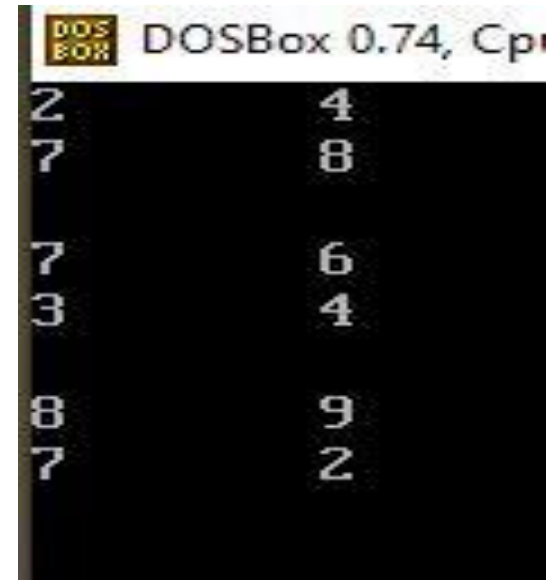
```
int arr [3] [2] [2]={  
    { { 2,4 }, { 7,8 } } ,  
    { { 7,6 } ,{ 3,4 } } ,  
    { { 8,9 } , { 7,2 } }  
};
```

Memory map of 3-dimensional array



Print Elements of 3D array

```
for(k=0;k<3;k++)
{
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            printf("%d\t",arr[k][i][j]);
        }
        printf("\n");
    }
    printf("\n");
}
```



Program : Read elements of 3D array

```
for(k=0;k<3;k++)  
{  
    for(i=0;i<2;i++)  
    {  
        for(j=0;j<2;j++)  
        {  
            scanf("%d",&arr[k][i][j]);  
        }  
    }  
}
```


Static arrays vs Dynamic arrays

- The process of allocating memory at compile time known as static memory allocation and the array that receive static memory allocation are called static arrays.
- Allocate the memory at run time known as dynamic memory allocation and the array created at run time are called dynamic arrays.
- Dynamic arrays are created using memory management functions malloc ,calloc and realloc.

Thank You